

CS7602 and CS8602

Coursework Case Study

Using AI to Explore a Security Research Problem

Contents

1.	Introduction to the coursework case study.....	2
2.	Project theme.....	2
3.	Choosing a paper	2
4.	What “using AI” really means (and does not)	3
5.	Compute and AI budget.....	4
6.	Timeline and milestone.....	4
7.	Mid-term design document (double column, 3–5 pages).....	4
8.	Final deliverables	5
9.	Expected level of completion (CS7602 vs CS8602)	5

1. Introduction to the coursework case study

In this coursework, you will use AI tools to explore a real security research problem drawn from a recent paper. The coursework is offered to both Master's and PhD students.

You will:

- Choose a security research paper (2023–2025).
- Identify a (sub-)problem that you can turn into a candidate space (e.g., rules, prompts, heuristics, parameters, pipelines), evaluate automatically with a fixed evaluate()-style script, and score using clear metrics (e.g., accuracy, detection rate, coverage, cost, overhead, attack success).
- Build an AI-driven search loop where AI proposes many candidate solutions, your evaluator scores them, and you analyse both the final solution and the evolution of the search process.

This project is inspired by the AI-Driven Research for Systems (ADRS) idea from

*Cheng et al. (2025), “Barbarians at the Gate: How AI is Upending Systems Research”,
arXiv:2510.06189.*

The emphasis is on clarity of problem formulation, instrumentation, and interpretation, not on building massive systems.

By the end of this project, you should be able to:

- Read and understand a modern security research paper at a systems or empirical level.
- Translate a research idea into a well-defined candidate space, a robust evaluator, and meaningful metrics.
- Use AI tools to search over designs, not just to generate one-off code.
- Critically analyse both what solution you ended up with and how the search process behaved (good/bad candidates, failure modes, surprises).

All experiments must be sandboxed, legal, and ethical.

2. Project theme

Your project must:

- Be rooted in a security or privacy problem (broadly interpreted: system security, network security, software security, privacy, cryptography in practice, LLM security, etc.).
- Be based on a recent paper (2023–2025) from a strong venue. Examples include: USENIX Security, IEEE S&P (“Oakland”), NDSS, CCS, ICSE, ISSTA, ASE, FSE, or a strong preprint / arXiv paper with sufficient technical detail.
- Identify a sub-problem where you can vary a candidate (policy, configuration, prompt, rule set, schedule, parameters, etc.), implement a fixed evaluator that scores each candidate automatically, and define one or more meaningful metrics (e.g., attack success rate, detection precision/recall, performance overhead, number of alerts, cost).

You do not need to re-implement an entire paper. A focused, well-instrumented sub-problem is ideal.

3. Choosing a paper

You are responsible for selecting a paper that is technically feasible given the time and available resources.

Before committing, check:

- Feasibility
 - You can run or re-implement at least a small version locally (no large cluster required).
 - There is code and/or data available, or the setting is simple enough to simulate.
- Clear candidate space
 - There is something you can vary repeatedly (rules, prompts, parameters, system configurations, etc.).
- Automatic evaluation
 - You can write a script like:


```
def evaluate(candidate) -> dict:
    # run experiment
    # return metrics, e.g. {"success_rate": ..., "cost": ...}
```
- Scope
 - The environment can be toy but honest: a small dataset, reduced configuration, or simple simulator is fine as long as the metric is meaningful.

Submit your choice

By Week 2, submit your chosen paper (title, link) and a 3–4 sentence justification explaining why it is security-related and why it is feasible for this course (local setup, data/code, clear candidate/evaluator/metric).

4. What “using AI” really means (and does not)

You must use AI as part of a search process, not just as a one-off assistant. Your pipeline should include:

- Candidate representation
 - How you encode a candidate (string, JSON, config file, rule list, prompt template, code snippet, etc.).
- Fixed evaluator
 - A deterministic (or controlled-random) `evaluate(candidate)` that runs your experiment and returns metrics.
 - Once you start running experiments, this evaluator should not change in major ways.
- AI-driven search loop
 - Use an LLM or other AI tool to generate initial candidates and/or iteratively modify candidates, using evaluation results as feedback.
 - You should run many candidates (at least tens; often more).
- Selection and analysis
 - Decide what “good” means (single metric or multi-objective).
 - Analyse best candidates, typical failures, and how performance changed over the search.

Clearly sufficient AI usage (examples)

- An LLM generates and iteratively refines IDS rules for a labelled dataset; you evaluate each rule set and feed back metrics to improve the next batch.
- An LLM proposes different prompt-based attack strategies on an LLM; you measure attack success on a fixed benchmark and explore how the strategy evolves.
- An LLM edits fuzzing (testing) strategies or input grammars; you track coverage and crashes as the search progresses.

Clearly insufficient AI usage

- Using an LLM once to generate boilerplate code or a single configuration.
- Asking AI “what threshold should I use?” and trying one value.
- Only using AI to help you write the evaluator, but never to propose or refine candidates.

Rule of thumb: if an external observer watches your logs, they should see many AI-proposed candidates being evaluated, not just a single suggestion.

5. Compute and AI budget

Practical resource budget (guideline)

- Assume you can reasonably afford tens to low hundreds of calls to evaluate(candidate), and tens to low hundreds of LLM calls, depending on prompt size and model.
- Design your search so that each evaluation is not too expensive and you can still explore an interesting part of the search space.

6. Timeline and milestone

Week 0–2: Paper and problem

- Read several candidate papers.
- Decide on a paper and sub-problem.
- Submit paper title and link, plus a short feasibility justification.

Week 3–4: Candidate and evaluator prototype

- Define candidate representation, initial metrics, and evaluator design.
- Implement a minimal `evaluate()` that runs on at least one dummy candidate.
- Submit a short proof of an “end-to-end run” (e.g., log, screenshot, or description).

Week 5–6: Midterm design document

- Finalise problem statement, candidate/evaluator/metrics, AI/search plan, risks and fallback.
- Submit the midterm design document (3–5 pages).
- Deliver an 8–10 minute presentation summarising the paper context, chosen (sub-)problem, candidate/evaluator/metrics design, AI/search approach, feasibility, and risks/fallback.

Week 7–10: Full implementation and search

- Implement full evaluator and search loop.
- Run experiments, collect data, and iterate.
- Start drafting your one-page case study.

Week 11–12: Finalise and submit

- Clean up code and README.
- Finalise the one-page case study.

7. Mid-term design document (double column, 3–5 pages)

Your document should cover:

- Paper context

- Core problem and threat model from the paper.
 - Which part you are focusing on.
- Your project problem
 - The precise sub-problem you chose.
 - Why it is interesting and non-trivial.
- Candidates, evaluator, and metrics
 - What a candidate looks like (with examples).
 - How evaluate(candidate) works (inputs, environment, outputs).
 - What metrics you will report and why they matter.
- Scope and assumptions
 - What you deliberately leave out (e.g., only one protocol, toy dataset).
 - Threat model assumptions.
- AI/search strategy
 - How you will generate initial candidates.
 - How AI will propose or edit candidates (prompt structure, feedback used).
 - Any selection / pruning rules (best-of-k, diversity, thresholds, etc.).
- Feasibility and risks
 - Rough resource needs (compute, LLM calls, storage).
 - Main risks and your fallback plan.

Think of this document as your project contract. You can still adapt later, but major changes should be discussed.

8. Final deliverables

You will submit:

1. Code + README
 - A repository containing the evaluator, AI search scripts, and configuration (prompts, config files, etc.).
 - The README should explain how to install dependencies, how to run a small demo, and how to reproduce key findings (on a reduced scale).
2. One-page case study (single column)
 - A concise, practitioner-friendly write-up covering: the problem setup, solution generator, evaluator and metrics, results, search behaviour, and lessons for practitioners.

9. Expected level of completion (CS7602 vs CS8602)

All projects are individual.

CS8602 (PhD students). You are expected to deliver a working end-to-end pipeline (candidate → evaluator → AI search) and to run a meaningful set of experiments (e.g., tens of candidates)

generating quantitative results. Projects that do not reach an end-to-end evaluation will lose a substantial portion of the mark.

CS7602 (Master's students). You are encouraged, but not strictly required, to reach a fully working end-to-end pipeline. If your system is only partially working by the deadline, you can still obtain marks by:

- clearly documenting what is implemented,
- explaining what you tried to debug or simplify the setup, and
- providing a critical analysis of why the current approach did not fully succeed and what you would do next.